CL-2006 Operating System Lab

Code Explanation Guide

Lab 06: Inter-Process Communication (IPC)
File Descriptors, Pipes & Named Pipes (FIFO)

Complete Line-by-Line Code Walkthrough
with Theoretical Concepts & Quiz Preparation

7 Examples Fully Explained:

Example 1: File Descriptors in Action
Example 2: Redirecting stdout with `dup2()`
Example 3: File Descriptors Survive `fork()`
Example 4: Basic Pipe – Parent Writes, Child Reads
Example 5: Two-Way Communication with a Single Pipe
Example 6: FIFO Writer Program
Example 7: FIFO Reader Program

National University of Computer & Emerging Sciences
Computer Science Department · Spring 2025

Contents

1 Key Theoretical Concepts – Lab 06 **2**
1.1 What is Inter-Process Communication (IPC)? . . . 2
1.2 What is a File Descriptor (FD)? . . . 2
1.3 The Three Standard File Descriptors . . . 2
1.4 How the Kernel Assigns File Descriptors . . . 3
1.5 System Calls Quick Reference . . . 3

2 Example 1: Seeing File Descriptors in Action (fd_demo.c) **4**
2.1 Line-by-Line Explanation . . . 4

3 Example 2: Redirecting stdout Using dup2() (dup2_demo.c) **7**
3.1 Line-by-Line Explanation . . . 7

4 Example 3: File Descriptors Survive fork() (fd_fork.c) **9**
4.1 Line-by-Line Explanation . . . 9

5 Unnamed Pipes – Theory Before Examples 4 & 5 **12**

6 Example 4: Basic Pipe – Parent Writes, Child Reads (unnamed_pipe.c) **13**
6.1 Line-by-Line Explanation . . . 14

7 Example 5: Two-Way Communication with a Single Pipe (unnamed_pipe2.c) **17**
7.1 Line-by-Line Explanation . . . 18

8 Named Pipes (FIFO) – Theory Before Examples 6 & 7 **21**

9 Example 6: FIFO Writer Program (fifo_write.c) **22**
9.1 Line-by-Line Explanation . . . 22

10 Example 7: FIFO Reader Program (fifo_read.c) **25**
10.1 Line-by-Line Explanation . . . 25

11 How Everything Connects – The Big Picture **28**

12 Common Mistakes to Avoid – Quiz Checklist **28**

13 Quick Reference Summary – Quiz Cheat Sheet **29**
13.1 File Descriptor Rules . . . 29
13.2 Pipe Rules . . . 29
13.3 FIFO Rules . . . 29
13.4 Compilation . . . 30

Key Theoretical Concepts – Lab 06

What is Inter-Process Communication (IPC)?

OS Concept – Why IPC Exists

Every process has its own **virtual address space**. This means one process cannot directly read or write another process's memory – they are completely isolated. The OS kernel acts as an intermediary, providing controlled communication channels (pipes, FIFOs, message queues, shared memory) that processes can use to exchange data safely. **Isolation by default, communication by explicit request** is the key design principle.

In Lab 05, you learned `fork()` to create processes. But processes also need to **talk to each other**. IPC is the set of mechanisms for that. Lab 06 covers three building blocks:

IPC Method	Direction	Related Procs?	Best Used For
Unnamed Pipe	Unidirectional	Yes (parent–child)	Simple data streaming
Named Pipe (FIFO)	Unidirectional	No (any processes)	Cross-process messaging
File Descriptors	Foundation	N/A	Basis of all IPC

What is a File Descriptor (FD)?

Core Concept – File Descriptor

A **file descriptor (FD)** is a small non-negative integer that the OS assigns to every open file, pipe, socket, or device. Think of it as a **ticket number at a bank** – when you open a file, the kernel gives you a number, and you use that number for all future operations (reading, writing, closing) on that resource.

Linux philosophy: **"Everything is a file."** Regular files, pipes, sockets, terminals, and hardware devices are all accessed through the same interface: file descriptors. The same `read()` and `write()` system calls work on all of them.

The Three Standard File Descriptors

Every process automatically starts with three FDs already open:

FD#	Name	C Constant	Purpose	Default
0	Standard Input	`STDIN_FILENO`	Reading input	Keyboard
1	Standard Output	`STDOUT_FILENO`	Normal output	Screen
2	Standard Error	`STDERR_FILENO`	Error messages	Screen

Tip

When you call `printf()`, it internally calls `write(1, ...)` – FD 1 (stdout). When you call `scanf()`, it calls `read(0, ...)` – FD 0 (stdin). Shell redirection `> file` is just the shell calling `dup2(fd, 1)` before running your program – now you know the mechanism!

How the Kernel Assigns File Descriptors

When you call `open()`, the kernel:

1. Searches for the **lowest available FD number** (0, 1, 2 are taken ⇒ starts from 3)
2. Creates an entry in the global open file table with: current offset, access mode, pointer to file's inode
3. Stores a reference in your process's FD table at that index
4. Returns the integer index to your program

OS Concept – FD Table and fork()

The FD table is **per-process**, but the open file descriptions in the kernel are **shared**. After `fork()`, the child gets a COPY of the parent's entire FD table. Both parent and child now have FDs pointing to the *same* underlying kernel objects. This is exactly how pipes work – create pipe (two FDs), call `fork()`, both inherit both ends, each closes the end it doesn't use.

System Calls Quick Reference

System Call	Header	Purpose
`open(path, flags, mode)`	`<fcntl.h>`	Open file/device/FIFO, returns FD
`close(fd)`	`<unistd.h>`	Close FD, free resource
`read(fd, buf, n)`	`<unistd.h>`	Read up to n bytes from FD
`write(fd, buf, n)`	`<unistd.h>`	Write n bytes to FD
`dup(fd)`	`<unistd.h>`	Duplicate FD
`dup2(oldfd, newfd)`	`<unistd.h>`	Redirect newfd to point where oldfd points
`pipe(int fd[2])`	`<unistd.h>`	Create pipe: `fd[0]`=read, `fd[1]`=write
`mkfifo(path, mode)`	`<sys/stat.h>`	Create named pipe at given path
`unlink(path)`	`<unistd.h>`	Delete FIFO/file from filesystem
`fork()`	`<unistd.h>`	Create child (inherits all FDs)
`wait(NULL)`	`<sys/wait.h>`	Wait for child to finish
`exit(status)`	`<stdlib.h>`	Terminate process

Example 1: Seeing File Descriptors in Action (fd_demo.c)

What This Example Teaches

How the kernel assigns file descriptor numbers sequentially starting from the lowest available integer. Also demonstrates that closing an FD "frees" that number for reuse by the next `open()` call.

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    printf("stdin  = %d\n", STDIN_FILENO);
    printf("stdout = %d\n", STDOUT_FILENO);
    printf("stderr = %d\n", STDERR_FILENO);

    int fd1 = open("file_a.txt", O_CREAT | O_WRONLY, 0644);
    int fd2 = open("file_b.txt", O_CREAT | O_WRONLY, 0644);
    int fd3 = open("file_c.txt", O_CREAT | O_WRONLY, 0644);
    printf("file_a = %d, file_b = %d, file_c = %d\n", fd1, fd2,
       fd3);

    close(fd2);

    int fd4 = open("file_d.txt", O_CREAT | O_WRONLY, 0644);
    printf("file_d = %d  (reused FD 4!)\n", fd4);

    close(fd1); close(fd3); close(fd4);
    return 0;
}
```

Listing 1: fd_demo.c – Seeing file descriptors in action

Line-by-Line Explanation

`#include <stdio.h>`

Includes the standard input/output library. Provides `printf()` for printing to the terminal.

`#include <fcntl.h>`

Includes the file control header. Provides the `open()` system call and its flag constants such as `O_CREAT`, `O_WRONLY`, and `O_RDONLY`.

`#include <unistd.h>`

Includes the POSIX API header. Provides `close()`, `read()`, `write()`, `dup()`, `dup2()`, `fork()`, and the standard FD constants `STDIN_FILENO`, `STDOUT_FILENO`, `STDERR_FILENO`.

```
printf("stdin = %d\n", STDIN_FILENO);
```

Prints the value of the constant `STDIN_FILENO`, which is always **0**. This is the file descriptor number for standard input (keyboard by default). The constant is defined in `<unistd.h>`.

```
printf("stdout = %d\n", STDOUT_FILENO);
```

Prints `STDOUT_FILENO`, which is always **1**. This is the FD for standard output (terminal screen). Every `printf()` call internally does `write(1, ...)`.

```
printf("stderr = %d\n", STDERR_FILENO);
```

Prints `STDERR_FILENO`, which is always **2**. This is the FD for standard error output. Functions like `perror()` write to this FD.

```
int fd1 = open("file_a.txt", O_CREAT | O_WRONLY, 0644);
```

Opens (or creates) `file_a.txt` for writing.
`O_CREAT` – create the file if it does not already exist.
`O_WRONLY` – open in write-only mode.
`0644` – permissions: owner can read+write (6), group can read (4), others can read (4). This is the standard octal permission format.
The kernel searches for the **lowest free FD number**. Since 0, 1, 2 are taken, it returns **3**.

```
int fd2 = open("file_b.txt", O_CREAT | O_WRONLY, 0644);
```

Same as above for `file_b.txt`. FD 3 is now taken, so the kernel returns **4**.

```
int fd3 = open("file_c.txt", O_CREAT | O_WRONLY, 0644);
```

Opens `file_c.txt`. FDs 3 and 4 are taken, so the kernel returns **5**.

```
printf("file_a = %d, file_b = %d, file_c = %d\n", fd1, fd2, fd3);
```

Prints the FD numbers assigned. You will see: `file_a = 3, file_b = 4, file_c = 5`. This confirms the sequential lowest-available-FD rule.

```
close(fd2);
```

Closes FD 4 (the FD assigned to `file_b.txt`). This **frees FD number 4**. The kernel marks slot 4 in this process's FD table as available. The actual file on disk is not deleted – just the file descriptor is released.

```
int fd4 = open("file_d.txt", O_CREAT | O_WRONLY, 0644);
```

Opens `file_d.txt`. The kernel again searches for the lowest available FD. FD 3 is taken (file_a), FD **4 is now free** (we just closed it), so the kernel **reuses FD 4**. This demonstrates the lowest-available rule precisely.

```
printf("file_d = %d (reused FD 4!)\n", fd4);
```

Confirms that `fd4 = 4`. The kernel reused the freed slot.

```
close(fd1); close(fd3); close(fd4);
```

Closes all remaining open file descriptors. Always close FDs when done. Each process has a limit (typically 1024 FDs). Forgetting to close causes **resource leaks**.

Expected Output

```
stdin  = 0
stdout = 1
stderr = 2
file_a = 3, file_b = 4, file_c = 5
file_d = 4  (reused FD 4!)
```

Why This Matters for IPC

`dup2()` (Example 2) and pipes (Examples 4 & 5) exploit this lowest-available rule. To redirect stdout (FD 1) to a file: close FD 1, then open a file – the kernel gives you FD 1. Or use `dup2()` which does this atomically and safely.

Example 2: Redirecting stdout Using dup2() (dup2_demo.c)

What This Example Teaches

How to use `dup2()` to redirect standard output (FD 1) to a file, so all subsequent `printf()` output goes to the file instead of the terminal. This is exactly how the shell implements the `>` operator.

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    printf("This goes to the terminal.\n");

    int fd = open("output.txt", O_CREAT | O_WRONLY | O_TRUNC,
        0644);

    dup2(fd, STDOUT_FILENO);
    close(fd);

    printf("This goes to the file!\n");
    printf("So does this.\n");

    return 0;
}
```

Listing 2: dup2_demo.c – Redirecting stdout to a file using dup2()

Line-by-Line Explanation

`printf("This goes to the terminal.\n");`

At this point, FD 1 (stdout) still points to the terminal. So this `printf()` writes to the terminal screen as normal. This line executes **before** we do any redirection.

`int fd = open("output.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);`

Opens (or creates) `output.txt` for writing.
`O_CREAT` – create if it does not exist.
`O_WRONLY` – write-only mode.
`O_TRUNC` – if the file already exists, truncate it to zero length (erase old content).
`0644` – standard read/write permissions.
The kernel assigns the next available FD (likely **3**) and stores it in `fd`.

`dup2(fd, STDOUT_FILENO);`

This is the **key line**. `dup2(oldfd, newfd)` does two things atomically:
Step 1: If `newfd` (FD 1 = stdout) is currently open, **close it** (disconnects from terminal).

Step 2: Make `newfd` (FD 1) point to the **same open file** as `oldfd` (our file FD 3). Result: FD 1 now points to `output.txt`. The terminal connection for stdout is gone. `STDOUT_FILENO` is the constant **1**.

`close(fd);`

Closes the original file descriptor `fd` (FD 3). We no longer need it because FD 1 already points to `output.txt`. Both FD 1 and FD 3 were pointing to the same file – we only need one of them now. Closing FD 3 does NOT close the file – it just removes one of the two references to it.

`printf("This goes to the file!\n");`

Now FD 1 points to `output.txt`. Since `printf()` internally calls `write(1, ...)`, this output goes to the **file**, not the terminal. The program does not "know" it was redirected – it still writes to FD 1 as usual.

`printf("So does this.\n");`

Also goes to `output.txt` for the same reason.

Expected Terminal and File Output

```
$ ./dup2_demo
This goes to the terminal.       <-- appears on screen

$ cat output.txt
This goes to the file!           <-- saved in file
So does this.
```

OS Concept – How the Shell Implements > Redirection

When you type `./program > output.txt` in the shell, the shell does exactly this:
1. Calls `open("output.txt", O_CREAT|O_WRONLY|O_TRUNC, 0644)` ⇒ gets FD 3
2. Calls `dup2(3, 1)` ⇒ FD 1 (stdout) now points to the file
3. Calls `close(3)` ⇒ clean up the extra FD
4. Calls `exec()` to run your program

Your program starts running with FD 1 already pointing to the file. It has no idea it was redirected. This is the beauty of the FD abstraction.

dup2 vs dup

`dup(fd)` duplicates `fd` onto the **lowest available** FD number.
`dup2(oldfd, newfd)` duplicates `oldfd` onto a **specific** FD number (`newfd`).
For I/O redirection you always want `dup2` because you need to target FD 0, 1, or 2 specifically.

Example 3: File Descriptors Survive fork() (fd_fork.c)

What This Example Teaches

The critical link between file descriptors and IPC: when `fork()` is called, the child process inherits **all** open file descriptors from the parent. Both parent and child can then use the same FD to write to the same file – or the same pipe.

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int fd = open("shared.txt", O_CREAT | O_WRONLY | O_TRUNC,
       0644);

    int pid = fork();

    if (pid > 0) {
        write(fd, "Parent wrote this\n", 18);
        wait(NULL);
        close(fd);
    }
    else if (pid == 0) {
        write(fd, "Child wrote this\n", 17);
        close(fd);
    }

    return 0;
}
```

Listing 3: fd_fork.c – File descriptors inherited across fork()

Line-by-Line Explanation

`#include <sys/wait.h>`

Provides the `wait()` system call, which makes the parent pause until a child process finishes. Needed to prevent a **zombie process** (a terminated child whose exit status has not been collected by the parent).

`int fd = open("shared.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);`

Opens `shared.txt` for writing **before** calling `fork()`. This is intentional and critical. The kernel assigns this process an FD (say FD 3) pointing to `shared.txt`. Because we open *before* fork, both the parent and child will inherit this same FD.

`int pid = fork();`

Creates a child process. From this single point, two processes exist. `fork()` does

three things:
– Returns the **child's PID** to the parent (`pid > 0`)
– Returns **0** to the child (`pid == 0`)
– Returns **-1** on failure
Crucially: the child gets a **copy of the parent's entire FD table**. So the child also has FD 3 pointing to `shared.txt` – the same kernel-level open file description, including the shared file offset.

```
if (pid > 0) {
```

Parent branch. Executes only in the parent process (where `fork()` returned child's PID, which is > 0).

```
write(fd, "Parent wrote this\n", 18);
```

The parent writes 18 bytes into `shared.txt` using FD 3. The kernel advances the **shared file offset** by 18 bytes. Because the offset is part of the shared open file description (not per-process), the child will see the advanced position.

```
wait(NULL);
```

Parent waits for the child to finish before continuing. The `NULL` argument means we don't care about the child's exit status – we just want to know when it's done. Without `wait()`, we'd get a **zombie process**.

```
close(fd);
```

Parent closes its copy of FD 3. The file is still open in the child (until the child also closes it).

```
else if (pid == 0) {
```

Child branch. Executes only in the child process (where `fork()` returned 0).

```
write(fd, "Child wrote this\n", 17);
```

The child also writes to `shared.txt` using the same FD 3. Because the file offset was advanced by the parent's write (18 bytes), the child's write goes **after** the parent's text. They do not overwrite each other.

```
close(fd);
```

Child closes its copy of FD 3.

Expected Output in shared.txt

```
$ cat shared.txt
Parent wrote this
Child wrote this
```

OS Concept – Why This Works: Shared File Offset

After `fork()`, parent and child both hold FDs pointing to the **same open file description** in the kernel. The file offset (current read/write position) is part of this shared description. So when parent writes 18 bytes and advances the offset to 18, the child's subsequent write starts at offset 18 automatically. This shared-offset behavior is what makes pipes work: both ends reference the same kernel buffer.

Always Close FDs!

Each process has a limit of 1024 file descriptors. Forgetting to `close()` causes **resource leaks**. In pipe-based IPC specifically, leaving a pipe's write end open in a process that should only be reading causes the reader's `read()` to **block forever** – it waits for data from a write end that will never close.

Unnamed Pipes – Theory Before Examples 4 & 5

What is an Unnamed Pipe?

An unnamed pipe is the **simplest IPC mechanism** in Linux. It is a kernel buffer with exactly **two file descriptors**: one for reading (`pipefd[0]`) and one for writing (`pipefd[1]`). Data flows in **one direction only** – from the write end to the read end. Think of a garden hose: water flows from tap (write end) to plant (read end), never backward.

Array Index	FD Role	Direction	Same as...
`pipefd[0]`	Read end	Data comes OUT	`read(fd, buf, ...)`
`pipefd[1]`	Write end	Data goes IN	`write(fd, buf, ...)`

OS Concept – Automatic Blocking (Built-in Synchronization)

A pipe is a kernel buffer (typically 64 KB on Linux). The kernel manages blocking automatically:
If pipe is empty: `read()` blocks until data is available.
If pipe is full: `write()` blocks until space is freed.
This means no extra synchronization code is needed – the pipe itself provides natural coordination between the writer and reader.

The Standard Pipe + Fork Pattern:

1. Call `pipe(pipefd)` to create the pipe (two FDs in kernel)
2. Call `fork()` – both parent and child now inherit both FDs
3. Each process **closes the end it does not use**
4. One writes (`write(pipefd[1], ...)`), the other reads (`read(pipefd[0], ...)`)

Always Close Unused Pipe Ends!

After `fork()`, if the parent is only writing, it must `close(pipefd[0])` (the read end). If the child is only reading, it must `close(pipefd[1])` (the write end). If you skip this, the reader's `read()` will **never see EOF** – it will block forever waiting for data from a write end that exists in the parent but will never be used. This is one of the most common pipe bugs.

Example 4: Basic Pipe – Parent Writes, Child Reads (unnamed_pipe.c)

What This Example Teaches

The fundamental pipe pattern: parent creates a pipe, forks a child, parent writes a message into the pipe, child reads the message. Demonstrates proper closing of unused pipe ends.

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void) {
    int pipefd[2];
    char buffer[15];

    pipe(pipefd);
    int pid = fork();

    if (pid > 0) {
        close(pipefd[0]);
        printf("unnamed_pipe [INFO] Parent Process\n");
        write(pipefd[1], "Hello Mr.Linux", 15);
        close(pipefd[1]);
        wait(NULL);
    }
    else if (pid == 0) {
        close(pipefd[1]);
        sleep(2);
        printf("unnamed_pipe [INFO] Child Process\n");
        read(pipefd[0], buffer, sizeof(buffer));
        printf("Received: %s\n", buffer);
        close(pipefd[0]);
    }
    else {
        printf("unnamed_pipe [ERROR] fork failed\n");
    }
    return 0;
}
```

Listing 4: unnamed_pipe.c – Basic pipe: parent writes, child reads

Line-by-Line Explanation

```
#include <sys/types.h>
```

Provides the `pid_t` data type used for process IDs returned by `fork()`. Good practice to include whenever using `fork()`.

```
int pipefd[2];
```

Declares a two-element integer array to hold the two pipe file descriptors. After `pipe(pipefd)`, the kernel fills:
`pipefd[0]` = the read end FD
`pipefd[1]` = the write end FD

```
char buffer[15];
```

A character array to hold the received message. Size 15 matches the message "Hello Mr.Linux" (14 chars + null terminator). Used by the child to store the data read from the pipe.

```
pipe(pipefd);
```

Creates the unnamed pipe. **Must be called BEFORE** `fork()`. The kernel creates a buffer in memory and fills `pipefd[0]` and `pipefd[1]` with two valid FD numbers. If we called `pipe()` after `fork()`, each process would get its own separate pipe – they would not share the same channel.

```
int pid = fork();
```

Creates the child process. Both parent and child now have **copies of both pipe FDs** in their FD tables, both pointing to the same kernel pipe buffer. Now we need to set up who reads and who writes.

```
if (pid > 0) { // PARENT
```

The parent block. `pid > 0` means we are in the parent (received child's PID). The parent will **write** into the pipe.

```
close(pipefd[0]);
```

Parent closes the **read end** of the pipe. The parent only writes; it will never read. Closing unused ends is mandatory – it prevents deadlocks and signals EOF correctly to the reader.

```
printf("unnamed_pipe [INFO] Parent Process\n");
```

Parent announces its role. This prints to the terminal (FD 1 is still the terminal here).

```
write(pipefd[1], "Hello Mr.Linux", 15);
```

Writes the string "Hello Mr.Linux" (15 bytes including the null terminator) into the pipe's write end. The data goes into the kernel's pipe buffer. The child can now read it. If the buffer were full, this would block.

`close(pipefd[1]);`

Parent closes the **write end** after finishing writing. This is important: when the child calls `read()` and the pipe is empty, it will get EOF (return 0) only when **all write ends are closed**. If the parent did not close this, the child's `read()` might block waiting for more data even after the message was sent.

`wait(NULL);`

Parent waits for the child to finish. Without this, the parent would exit first, and the child might become an orphan. `NULL` means we don't need the child's exit status.

`else if (pid == 0) { // CHILD`

The child block. `pid == 0` means we are in the child process.

`close(pipefd[1]);`

Child closes the **write end** of the pipe. The child only reads; it will never write. Again, closing unused ends is mandatory.

`sleep(2);`

Child sleeps for 2 seconds to simulate processing delay. During this sleep, the parent has likely already written the message into the pipe (the message sits in the kernel buffer waiting). This shows that the pipe **buffers data** – the writer doesn't have to wait for the reader to be ready.

`read(pipefd[0], buffer, sizeof(buffer));`

Child reads from the pipe's read end into `buffer`. `sizeof(buffer) = 15`. If the pipe were empty at this moment, `read()` would **block** until data arrives. Since the parent already wrote before the child woke up, data is already waiting.

`printf("Received: %s\n", buffer);`

Prints the received message. `%s` treats `buffer` as a null-terminated string.

`close(pipefd[0]);`

Child closes the read end after finishing. All FDs are now closed in both processes.

`else { printf("unnamed_pipe [ERROR] fork failed\n"); }`

Error handler: if `fork()` returns -1, it failed (system is out of resources or hit process limit). Always handle the error case.

Expected Output

```
unnamed_pipe [INFO] Parent Process
unnamed_pipe [INFO] Child Process
Received: Hello Mr.Linux
```

(The 2-second `sleep()` means "Child Process" appears 2 seconds after "Parent Process")

Built-in Synchronization

Notice the child's `read()` automatically waits until the parent writes data. The kernel handles all the blocking for you. No semaphores or mutexes needed – the pipe provides natural synchronization between the two processes.

Example 5: Two-Way Communication with a Single Pipe (unnamed_pipe2.c)

What This Example Teaches

How to achieve **bidirectional communication** by reusing a single pipe – parent writes first, child reads then replies, parent reads the reply. Also teaches the critical `exit(0)` rule in child processes.

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main(void) {
    int pipefd[2];
    char buffer[15];

    pipe(pipefd);
    int pid = fork();

    if (pid > 0) {
        printf("unnamed_pipe [INFO] Parent Process\n");
        write(pipefd[1], "Hello Child", 12);
        close(pipefd[1]);
    }
    else if (pid == 0) {
        sleep(2);
        printf("unnamed_pipe [INFO] Child Process\n");
        read(pipefd[0], buffer, sizeof(buffer));
        close(pipefd[0]);
        printf("Received: %s\n", buffer);

        write(pipefd[1], "Hello Parent", 13);
        close(pipefd[1]);
        exit(0);
    }
    else {
        printf("unnamed_pipe [ERROR] fork failed\n");
    }

    wait(NULL);
    read(pipefd[0], buffer, sizeof(buffer));
    close(pipefd[0]);
    printf("Reply from Child: %s\n", buffer);

    return 0;
}
```

Listing 5: unnamed_pipe2.c – Two-way communication with one pipe

Line-by-Line Explanation

```
#include <stdlib.h>
```

Provides `exit()` function. Required here because the child must call `exit(0)` to terminate cleanly and prevent falling through to the parent's code after the `if/else` block.

```
pipe(pipefd); int pid = fork();
```

Same pattern as Example 4: create the pipe before forking so both processes inherit both FD ends. After `fork()`, both parent and child have `pipefd[0]` (read) and `pipefd[1]` (write).

```
if (pid > 0) { // PARENT writes first
```

Parent block. The parent's role here is to **write first**, then later read the child's reply.

```
write(pipefd[1], "Hello Child", 12);
```

Parent writes "Hello Child" (12 bytes) into the pipe. The data sits in the kernel buffer.

```
close(pipefd[1]);
```

Parent closes the write end **immediately after writing**. This is the signal to the child that no more data is coming from the parent. When the child reads the pipe and it becomes empty, the child can detect EOF. **Note:** The parent closes the write end here, but the child still has a copy of the write end open – the pipe is not yet fully closed.

```
else if (pid == 0) { // CHILD
```

Child block.

```
sleep(2);
```

Child waits 2 seconds. The parent has already written by this point. The data is buffered in the kernel waiting for the child.

```
read(pipefd[0], buffer, sizeof(buffer));
```

Child reads the parent's message from the pipe into `buffer`. Reads up to 15 bytes. Gets "Hello Child\0".

```
close(pipefd[0]);
```

Child closes the read end after reading the parent's message. The child is done reading for now.

```
printf("Received:  %s\n", buffer);
```

Child prints the message it received from the parent.

```
write(pipefd[1], "Hello Parent", 13);
```

Child writes its **reply** – "Hello Parent" (13 bytes) – back into the pipe's write end. Since `pipefd[1]` is still open in the child (the child never closed it yet), the child can write here. The parent will read this later.

```
close(pipefd[1]);
```

Child closes the write end after sending the reply. Now all of the child's pipe FDs are closed.

```
exit(0);
```

Critical line! The child calls `exit(0)` to terminate. Without this, the child would fall through past the `else if` block and execute the parent's code below (`wait()`, `read()`, etc.), causing the reply to be printed **twice** and other unexpected behavior. **Always use `exit(0)` at the end of a child process branch.**

```
wait(NULL);
```

Only the parent reaches this line (the child already exited). Parent waits for the child to finish, then proceeds to read the child's reply.

```
read(pipefd[0], buffer, sizeof(buffer));
```

Parent reads the child's reply "Hello Parent" from the pipe. This works because the child wrote to `pipefd[1]` before exiting, and those bytes are still in the pipe buffer waiting for the parent to read.

```
close(pipefd[0]);
```

Parent closes the read end. All cleanup complete.

```
printf("Reply from Child:  %s\n", buffer);
```

Parent prints the child's reply.

Expected Output

```
unnamed_pipe [INFO] Parent Process
unnamed_pipe [INFO] Child Process
Received: Hello Child
Reply from Child: Hello Parent
```

The Critical Bug – Missing exit(0)

Without `exit(0)` in the child, BOTH parent and child execute the code after the `if/else` block. The child would call `wait(NULL)` (which would fail since the child has no children), then `read()` (which might get the same data or block), and print "Reply from Child" a second time. This is one of the **most common mistakes with fork()**.

Better Approach: Use Two Pipes

This single-pipe bidirectional trick is fragile because timing matters – the sequence of reads and writes must be perfectly coordinated. For robust bidirectional IPC, use **two separate pipes**:

- `pipe_pc[2]` – parent writes, child reads (parent→child direction)
- `pipe_cp[2]` – child writes, parent reads (child→parent direction)

This is the pattern used in Assignment 03 and is the professional standard.

Named Pipes (FIFO) – Theory Before Examples 6 & 7

What is a Named Pipe (FIFO)?

An unnamed pipe only works between **related processes** (parent–child) because the FDs are shared only through `fork()` inheritance. A **named pipe (FIFO – First In, First Out)** solves this: it is a special file on the filesystem with a real path (e.g., `/tmp/myfifo`). Any process that knows the path can open it, regardless of relationship.

Real-world analogy: A named pipe is like a **letterbox**. Process A drops a letter in; Process B picks it up. They don't need to know each other – they just both know the letterbox address.

Feature	Unnamed Pipe	Named Pipe (FIFO)
Creation	`pipe()` system call	`mkfifo()` system call
Filesystem	No – kernel memory only	Yes – visible as a file
Process relationship	Must be parent–child	Any processes
Identification	Inherited FDs	File path (e.g., `/tmp/myfifo`)
Persistence	Gone when both ends close	Stays until `unlink()` is called
Permissions	Inherited from parent	Set at creation (`0666`)

Critical Synchronization Property of Named Pipes

When you call `open()` on a FIFO for **reading** (`O_RDONLY`), it **BLOCKS** until another process opens the same FIFO for **writing** (`O_WRONLY`), and vice versa. Both sides must be connected before any data can flow. This is automatic synchronization – no extra code needed.

Example 6: FIFO Writer Program (fifo_write.c)

What This Example Teaches

How to create a named pipe with `mkfifo()` and write data to it. The writer blocks on `open()` until a reader connects – showing FIFO's built-in connection synchronization.

```c
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <fcntl.h>
#include <unistd.h>

int main(void) {
    int fd;
    char buffer[] = "Hello from the writer process!";

    mkfifo("/tmp/myfifo", 0666);

    printf("Writer: Waiting for reader to connect...\n");
    fd = open("/tmp/myfifo", O_WRONLY);
    printf("Writer: Reader connected! Sending message.\n");

    write(fd, buffer, sizeof(buffer));
    close(fd);

    printf("Writer: Message sent. Exiting.\n");
    return 0;
}
```

Listing 6: fifo_write.c – FIFO writer: create and write to named pipe

Line-by-Line Explanation

`#include <sys/stat.h>`

Provides the `mkfifo()` function and related mode constants. Required for creating named pipes. Also provides the `mode_t` type used for permission bits.

`#include <sys/types.h>`

Provides fundamental data types used by the system calls, such as `mode_t` and `size_t`. Standard include for any program using POSIX system calls.

`int fd;`

Declares the file descriptor variable. After `open()` succeeds, this holds the FD for the FIFO – just like any regular file descriptor.

```
char buffer[] = "Hello from the writer process!";
```

Declares and initializes the message to send. The size is automatically set by the compiler to fit the string including its null terminator. `sizeof(buffer)` will return 31 (30 characters + 1 null byte).

```
mkfifo("/tmp/myfifo", 0666);
```

Creates a named pipe (FIFO) at the path `/tmp/myfifo`.
`/tmp/myfifo` – the full filesystem path where the FIFO file will appear. Any process knowing this path can use the pipe.
`0666` – permissions: owner, group, and others all get read+write (rw-rw-rw-). The actual permissions may be modified by the process's `umask`.
If the FIFO already exists, `mkfifo()` returns -1 (but the program continues since we don't check the return value here).

```
printf("Writer: Waiting for reader to connect...\n");
```

Prints a status message before calling `open()`. This message appears on the terminal immediately. The next line will block, so this message tells the user why the program appears to hang.

```
fd = open("/tmp/myfifo", O_WRONLY);
```

Opens the FIFO for writing.
Key behavior: This call **BLOCKS** until a reader opens the same FIFO with `O_RDONLY`. The writer process is paused here until the reader connects. This is named pipe's automatic connection synchronization – both ends must be ready before any data flows.
Once a reader opens the FIFO, `open()` returns an FD number (e.g., 3) and execution continues.

```
printf("Writer: Reader connected! Sending message.\n");
```

This line only executes **after the reader has connected**. Confirms that both sides are now ready to communicate.

```
write(fd, buffer, sizeof(buffer));
```

Writes the message into the FIFO. `sizeof(buffer) = 31` bytes. The data flows through the kernel's FIFO buffer to the reader. **Same `write()` call as for files and unnamed pipes** – the FD abstraction unifies everything.

```
close(fd);
```

Closes the writer's end of the FIFO. When the reader tries to read after this, it will eventually receive EOF (the pipe is empty and the write end is closed).

```
printf("Writer: Message sent. Exiting.\n");
```

Final status message. The writer's work is done.

How to Run This

```
# Compile:
gcc fifo_write.c -o write
gcc fifo_read.c  -o read

# Run in TWO separate terminals:
# Terminal 1:           Terminal 2:
./write                 ./read
Writer: Waiting...      Reader: Waiting...
(both block until the other opens)
Writer: Connected!      Reader: Connected!
Writer: Sent. Exiting.  Reader: Received: Hello from the writer process!
```

Running Writer Twice Without Cleanup

If you run the writer program a second time without deleting the FIFO, `mkfifo()` will fail (returns -1, `errno = EEXIST`) because the file already exists. The program will still work if you don't check the return value (since `open()` can open an existing FIFO). But for robust code, always check `mkfifo()`'s return value or call `unlink()` first.

Example 7: FIFO Reader Program (fifo_read.c)

What This Example Teaches

The complementary program to Example 6. The reader opens the same FIFO for reading, receives the message, and cleans up the FIFO file with `unlink()`. Run in a separate terminal at the same time as the writer.

```c
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <fcntl.h>
#include <unistd.h>

int main(void) {
    int fd;
    char buffer[512] = {0};

    printf("Reader: Waiting for writer to connect...\n");
    fd = open("/tmp/myfifo", O_RDONLY);
    printf("Reader: Writer connected! Reading message.\n");

    read(fd, buffer, sizeof(buffer));
    close(fd);

    printf("Reader: Received: %s\n", buffer);

    unlink("/tmp/myfifo");
    return 0;
}
```

Listing 7: fifo_read.c – FIFO reader: read from named pipe and clean up

Line-by-Line Explanation

`char buffer[512] = {0};`

Declares a 512-byte buffer for the incoming message and **initializes all bytes to 0**. The `= {0}` syntax zero-initializes the entire array. This is important: without it, the buffer contains garbage values. If the read data is shorter than 512 bytes, the remaining bytes stay as 0, ensuring the string is always null-terminated.

`printf("Reader: Waiting for writer to connect...\n");`

Status message before `open()`. This prints immediately to let the user know the program is running but waiting for the other side.

`fd = open("/tmp/myfifo", O_RDONLY);`

Opens the **same FIFO** at `/tmp/myfifo` for reading.
O_RDONLY – open in read-only mode.

Key behavior: This **BLOCKS** until the writer opens the FIFO with `O_WRONLY`. The reader and writer unblock each other simultaneously – once both call `open()`, both proceed. This is the FIFO's automatic handshake.
The reader does **not** call `mkfifo()` – the writer already created the FIFO, and the reader just opens the existing file.

```
printf("Reader: Writer connected! Reading message.\n");
```

Executes only after the writer has connected. Both programs print their "connected" messages around the same time (once both `open()` calls return).

```
read(fd, buffer, sizeof(buffer));
```

Reads up to 512 bytes from the FIFO into `buffer`. `sizeof(buffer) = 512`. If no data has arrived yet, this **blocks** until data is available. Once the writer sends the message, this returns with the bytes read. The return value (number of bytes read) is ignored here but in production code should be checked.

```
close(fd);
```

Closes the reader's FD for the FIFO. The FIFO file itself still exists on the filesystem – `close()` only releases the FD, not the file.

```
printf("Reader: Received: %s\n", buffer);
```

Prints the received message. `%s` prints the buffer as a null-terminated string. Since the writer sent "Hello from the writer process!" (30 chars + null), this prints exactly that string.

```
unlink("/tmp/myfifo");
```

Deletes the FIFO file from the filesystem. Unlike regular files, FIFOs persist after both programs exit unless explicitly deleted. `unlink()` removes the directory entry (the filename). If both programs exit without calling `unlink()`, the FIFO file remains and causes `mkfifo()` to fail next time (EEXIST error).

Expected Output in Both Terminals

```
Terminal 1 (Writer):                Terminal 2 (Reader):
Writer: Waiting for reader...       Reader: Waiting for writer...
(both block here until the other opens the FIFO)
Writer: Reader connected!           Reader: Writer connected!
Writer: Message sent. Exiting.      Reader: Received: Hello from the writer process!
```

OS Concept – FIFO Cleanup with unlink()

Named pipes persist on the filesystem because they are real files (you can see them with `ls -l`). After both programs exit, the FIFO file remains until `unlink()` is called. Calling `unlink()` removes the **filesystem entry** (name) – if any processes still have it open, the kernel data is kept until the last FD is closed. This is the

same deferred deletion behavior as regular files in Linux.

Cleanup Warning

Always call `unlink("/tmp/myfifo")` when done. From the command line: `rm /tmp/myfifo`. If you forget, the next run of `mkfifo()` will fail with "File exists" error and your program may behave unexpectedly.

How Everything Connects – The Big Picture

OS Concept – File Descriptors Tie Everything Together

File descriptors are the common thread through all of Lab 06:

Operation	FD Connection
`printf("hello")`	Calls `write(1, "hello", 5)` – writes to FD 1 (stdout)
`scanf("%d", &x)`	Calls `read(0, ...)` – reads from FD 0 (stdin)
`./prog > file.txt`	Shell uses `dup2(fd, 1)` to redirect FD 1 to file
`pipe(pipefd)`	Creates two FDs: `pipefd[0]` read, `pipefd[1]` write
`fork()` after pipe	Child inherits both pipe FDs – enables IPC
`mkfifo + open()`	Creates a named pipe file; `open()` returns an FD
`cat f | grep w`	Shell creates pipe, uses `dup2()` to wire FD 1 of `cat` to pipe write, FD 0 of `grep` to pipe read

How the Shell Pipe | Operator Works Internally

The shell command `cat file | grep word` uses exactly what you learned in Lab 06:

1. Shell calls `pipe(pipefd)` to create an unnamed pipe
2. Shell calls `fork()` to create Child 1 (for `cat`)
3. In Child 1: `dup2(pipefd[1], 1)` – redirect `cat`'s stdout to pipe write end
4. Child 1 calls `exec("cat", ...)` – `cat` writes to FD 1, data goes into pipe
5. Shell calls `fork()` to create Child 2 (for `grep`)
6. In Child 2: `dup2(pipefd[0], 0)` – redirect `grep`'s stdin to pipe read end
7. Child 2 calls `exec("grep", ...)` – `grep` reads from FD 0, gets data from pipe

Everything from Labs 05 (fork, exec, wait) and 06 (FDs, dup2, pipe) combines here!

Common Mistakes to Avoid – Quiz Checklist

Most Common Mistakes in Lab 06

1. **Creating pipe after fork():** If you call `pipe()` inside the child or parent block (after `fork()`), each process gets its own separate pipe – they won't share the same channel. Always call `pipe()` **before** `fork()`.
2. **Not closing unused pipe ends:** If the parent writes but doesn't close `pipefd[0]`, the child's `read()` will never see EOF and may block forever. **Rule:** Each process closes the end it doesn't use immediately after `fork()`.
3. **Missing `exit(0)` in child:** Without `exit(0)` at the end of the child's code block, the child falls through and executes the parent's code too. This is the classic fork() bug.
4. **Not calling `wait()` in parent:** Without `wait()`, the parent exits before the

child, creating a **zombie process** – a child that has finished but whose exit status hasn't been collected.

5. **Forgetting `unlink()` for FIFO:** Named pipes persist on the filesystem. If you don't call `unlink()`, the next `mkfifo()` call on the same path fails with EEXIST.
6. **Wrong byte count in write():** If you write 14 bytes but the reader's buffer is 15, the last byte in the buffer is garbage (from uninitialized memory). Always match byte counts or null-terminate properly.
7. **Buffer size mismatch:** In `read(pipefd[0], buffer, sizeof(buffer))`, the buffer must be large enough to hold all expected data. A too-small buffer truncates the message.

Quick Reference Summary – Quiz Cheat Sheet

File Descriptor Rules

- Kernel always assigns the **lowest available FD number**
- FDs 0, 1, 2 are always stdin, stdout, stderr
- `close(fd)` frees the FD number for reuse
- `fork()` gives child a **copy** of parent's FD table
- `dup2(oldfd, newfd)` – atomically redirect newfd to oldfd's target

Pipe Rules

- `pipefd[0]` = read end, `pipefd[1]` = write end
- Always call `pipe()` **before** `fork()`
- Each process closes the end it doesn't use
- `read()` blocks if pipe is empty; `write()` blocks if pipe is full
- `read()` returns 0 (EOF) when all write ends are closed
- Always `exit(0)` in child to prevent fall-through

FIFO Rules

- `mkfifo(path, 0666)` – creates the named pipe file
- `open(path, O_WRONLY)` **blocks** until reader opens; `O_RDONLY` blocks until writer opens
- Same `read()`/`write()` calls as unnamed pipes
- `unlink(path)` removes the FIFO file from filesystem
- Any two unrelated processes can use a FIFO (unlike unnamed pipes)

Compilation

```
gcc fd_demo.c       -o fd_demo
gcc dup2_demo.c     -o dup2_demo
gcc fd_fork.c       -o fd_fork
gcc unnamed_pipe.c  -o pipe1
gcc unnamed_pipe2.c -o pipe2
gcc fifo_write.c    -o write
gcc fifo_read.c     -o read
```

Listing 8: Compilation for Lab 06 programs

Good luck on your quiz!
Remember the golden rules: pipe() before fork() · close unused ends · exit(0) in child · unlink() for FIFO.